Compilation

CMSC 323: Design and implementation of programming languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in Programming Languages: Application and Interpretation.

Steps for evaluating an application

- Evaluate the function expression
- Evaluate the argument expression
- Bind the parameter to the argument value
- Extend the function environment to include the parameter-argument pair
- Evaluate the body of the function value using the extended environment

Exercise

Evaluate the following FAE expressions, showing where applicable the function expression, the argument expression and the results of the intermediate steps for evaluating an application

1. {lam {a b} {+ a b}}
2. {lam {a} {lam {b} {+ a b}}}
3. {{lam {a} {lam {b} {+ a b}}} 5}
4. {{{lam {a} {lam {b} {+ a b}}} 5} 6}

Exercise

What is the result of “interping” the following FAE expressions

1. {lam {a b} {+ a b}}

Solution

(funV 'a (lam 'b (plus (var 'a) (var 'b))) '#hash())

Exercise

What is the result of “interping” the following FAE expressions

1. {lam {a} {lam {b} {+ a b}}}

Solution

(funV 'a (lam 'b (plus (var 'a) (var 'b))) '#hash())

Exercise

What is the result of "interping" the following FAE expressions

1. {{lam {a} {lam {b} {+ a b}}} 5}

Solution

(funV 'b (plus (var 'a) (var 'b)) (hash 'a (numV 5)))

Reminders

- An interpreter takes a program and produces a result
- A compiler takes a program and produces a program
- A compiler bridges the gap between the programs we want to write and the programs we want to run

Why Compilers?

- Implementing technical redundancies (e.g. with's) in large languages
- Our available interpreter may only support a very restricted language
- Writing and debugging highly-optimized programs can be painful

Compiler Basics

- Interpreters relate two languages – an object and a meta language
- A compiler relates three languages – a source, target, and meta language

Compiler Basics

A compiler relates three languages – a source, target, and meta language

- The source language is the language of the inputs to the compiler
- It is akin to an interpreter's object language

Compiler Basics

A compiler relates three languages – a source, target, and meta language

- The target language is the language of the outputs of the compiler

Compiler Basics

A compiler relates three languages – a source, target, and meta language

- The meta language is the language the compiler itself is written in

Compiler Basics

Examples of language triples – source, target, meta

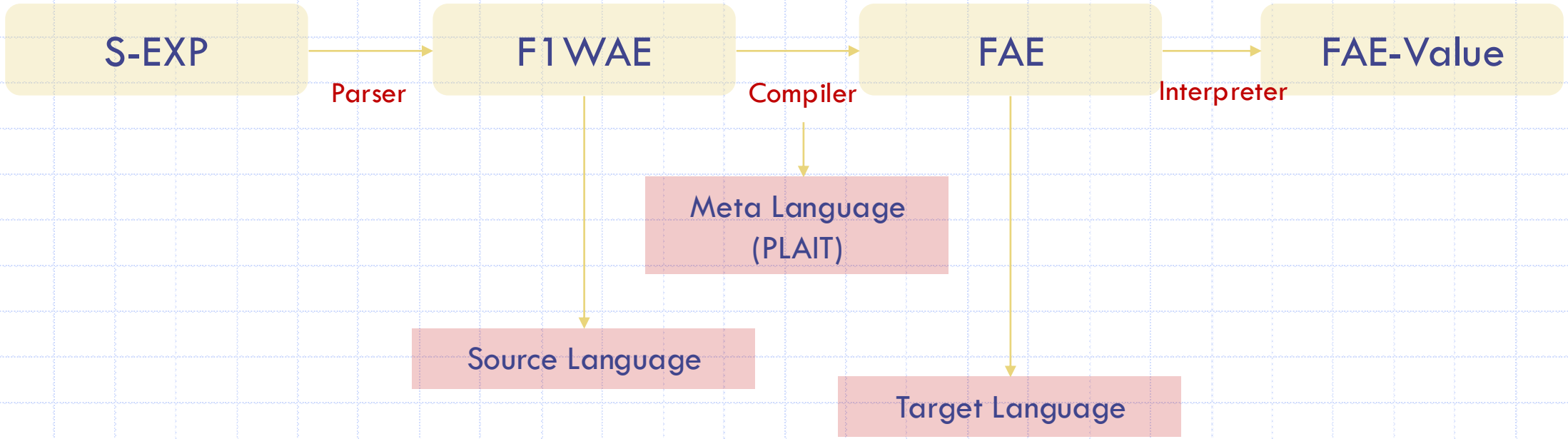
- GCC – C, machine code, C
- Java – Java, JVM byte code, Java

Compiler Basics

The compiler we will write in class today relates:

- Source: F1WAE – has with which is nicer to write in
- Target: FAE - no with, which is easier to implement
- Meta: PLAIT
- Our source and target languages are very close

Compiler languages: source, target, and meta languages



F1WAE vs FAE Grammar

```
<F1WAE> := <num>
         | {+ <F1WAE> <F1WAE>}
         | {- <F1WAE> <F1WAE>}
         | {with {<id><F1WAE>} <F1WAE>}
         | <id>
         | {lam {<id>} <F1WAE>}
         | {<F1WAE> <F1WAE>}
```

```
<FAE> := <num>
       | {+ <FAE> <FAE>}
       | {- <FAE> <FAE>}
       | <id>
       | {lam {<id>} <FAE>}
       | {<FAE> <FAE>}
```

F1WAE vs FAE

```
(define-type F1WAE
  [w-num (n : Number)]
  [w-plus (lhs : F1WAE) (rhs : F1WAE)]
  [w-sub (lhs : F1WAE) (rhs : F1WAE)]
  [w-var (id : Symbol)]
  [w-with (id : Symbol)
          (named-exp : F1WAE)
          (body : F1WAE)]
  [w-lam (param : Symbol) (body : F1WAE)]
  [w-app (func : F1WAE) (arg : F1WAE)])
```

```
(define-type FAE
  [num (n : Number)]
  [plus (lhs : FAE)(rhs : FAE)]
  [sub (lhs : FAE)(rhs : FAE)]
  [var (id : Symbol)]
  [lam (param : Symbol)(body : FAE)]
  [app (func : FAE) (arg : FAE)])
```

- Our source and target languages are very close
- They only make sense for demonstrating how compilers work
  - Sort of, since compiler translations are significantly more involved

Compiling F1WAE

with is turned into a lam and app

F1WAE

```
{with {x 3} x}
```

FAE

```
{{lam {x} x} 3}
```

Compiling F1WAE

with is turned into a lam and app

F1WAE

```
`{with {x 3}
     {with {y 4}
          {+ x y}}}
```

FAE

Compiling F1WAE

with is turned into a lam and app

F1WAE

```
`{with {x 3}
     {with {y 4}
          {+ x y}}}
```

FAE

```
{{lam {x} {{lam {y} {+ x y}} 4}} 3}
```

Compiling F1WAE

```
(test (compile (parse `{+ 1 2}))
      (parse-fae `{+ 1 2}))

(test (compile (parse `{with {x 3} x}))
      (parse-fae `{{lam {x} x} 3}))

(test (compile (parse `{+ 2
                          {with {y 7}
                                {+ y 3}}}))
      (parse-fae `{+ 2
                     {{lam {y} {+ y 3}}
                      7}}))
```

Compiling F1WAE

`w-num` and `w-var` translate directly

```
(compile : (F1WAE -> FAE))
(define (compile f1wae)
  (type-case F1WAE f1wae
    [(w-num n) (num n)]
    [(w-var id) (var id)]
. . .
))
```

Compiling F1WAE

What about `w-plus`, `w-sub`, `w-lam` and `w-app`?

Compiling F1WAE

We compile each expression in `w-plus, w-sub, w-lam` and `w-app` (structural recursion) since `w-with` may occur in them.

```
(compile : (F1WAE -> FAE))
(define (compile f1wae)
  (type-case F1WAE f1wae
. . .
    [(w-plus lhs rhs) (plus (compile lhs) (compile rhs))]
    [(w-sub lhs rhs) (sub (compile lhs) (compile rhs))]
    [(w-lam param body) (lam param (compile body))]
    [(w-app func arg) (app (compile func) (compile arg))]
. . .))
```

Compiling F1WAE

`w-with` are converted to `lam`s.

```
(compile : (F1WAE -> FAE))
(define (compile f1wae)
      (type-case F1WAE f1wae
        . . .
            [(w-with id named-exp body)
                 (app (lam id (compile body))
                 (compile named-exp))]))
```

Optimization with our Compiler

We will carry out a very basic optimization with our compiler – ***constant folding***.

Not perpetual folding, but folding of constants

Constant folding is recognizing and evaluating constants at compile time rather than at run-time

Constant Folding

2 + 2 = 4

The above is always true regardless of the rest of the program

So: if both operands of a plus or sub are constants, evaluate it at compile time and replace it with the result

{+ 2 2} => 4

Note: the check happens after the operands have been compiled, so folding cascades. In {- {+ 1 2} 3}, the inner {+ 1 2} folds to 3 first, so the outer sub sees two constants and folds to 0.

Constant Folding

Any time we see an add or sub, we see if we can constant fold

```
(compile : (F1WAE -> FAE))
(define (compile f1wae)
  (type-case F1WAE f1wae
        . . .
    [(w-plus lhs rhs) (try-constant-folding (plus (compile lhs) (compile rhs)))]
    [(w-sub lhs rhs) (try-constant-folding (sub (compile lhs) (compile rhs)))]
. . .
))
```

Constant Folding

an-fae is an FAE

```
(define (try-constant-folding an-fae)
  (type-case FAE an-fae
    [(plus lhs rhs) (if (and (num? lhs) (num? rhs))
                        (num (+ (num-n lhs) (num-n rhs)))
                        an-fae)]
    [(sub lhs rhs) (if (and (num? lhs) (num? rhs))
                       (num (- (num-n lhs) (num-n rhs)))
                       an-fae)]
    [else (error 'constant-folding "constant fold on plus and sub only")]))
```

Constant Folding

```
(test (compile (parse `{+ 1 2}))
      (parse-fae `3))

(test (compile (parse `{+ 1 x}))
      (parse-fae `{+ 1 x}))

(test (compile (parse `{f {+ 1 2}}))
      (parse-fae `{f 3}))

(test (compile (parse `{- {+ 1 2} 3}))
      (parse-fae `0))
```